

Building and Securing a REST API

MoMo SMS Transactions API

Ajang Akoi Arok Biar | Chol Deng

Repository: <https://github.com/Chol1000/dema3>

1. Introduction to API Security

An API is the way every client application gets to the data behind it, which means it is also the way anyone else would get to that data if it were left open. For a mobile money system that data is about as sensitive as it gets. It says how much money moved, who sent it, who received it, their phone numbers, and what the balance was afterwards. Someone who can reach an unprotected version of this API can read a customer's entire financial history, and if the write endpoints are open too, they can change it or delete it.

When people talk about securing an API they usually mean three different things, and it helps to keep them apart. **Authentication** is about working out who is making the request. **Authorization** is about deciding whether that person is allowed to do the particular thing they are asking for. **Confidentiality** is about stopping anyone else from reading the exchange while it crosses the network, which is what HTTPS gives you.

You need all three, because any one of them on its own leaves a hole. Checking a password carefully does not help if that password is being sent in a form anyone can read. Encrypting the connection does not help if every user who logs in is allowed to delete everything. This project covers the first of the three, using HTTP Basic Authentication, because that is what the assignment asked for. Every endpoint turns away requests that are not authenticated with a 401 response. Section 5 goes through where that approach falls short, which is further than we expected when we started.

2. System Overview and Data Parsing

The file we were given, `modified_sms_v2.xml`, is an SMS backup from an MTN Mobile Money account. It holds 1,691 `<sms>` elements. The catch is that the useful information is not sitting in neat XML fields. It is inside the body attribute as ordinary English sentences, the same text a person would read on their phone. So the parser cannot just pull out tags. It has to read meaning out of prose.

We handled that in `dsa/parse_xml.py` with a list of regular expressions, each one labelled with the kind of transaction it matches. Every SMS body is tested against the patterns in order, and the first one that matches decides the transaction type and pulls out the pieces we want: the amount, the fee, the new balance, the sender, the receiver, the account and the transaction id.

A few small things needed cleaning up along the way. Amounts appear in the messages written as "12,500", so they get their commas stripped and become numbers rather than staying as text. Dates get converted to ISO-8601. Some messages carry no readable time at all, and for those we fall back to the epoch timestamp stored on the element itself.

Here is how the 1,691 records break down:

Transaction type	Count	Transaction type	Count
payment_to_code_holder	658	otp	8
transfer_to_mobile	585	bank_transfer	6

Transaction type	Count	Transaction type	Count
bank_deposit	249	failed_transaction	5
incoming_money	63	withdrawal	3
bill_or_airtime_payment	53	reversal	2
third_party_payment	36	merchant_payment	2
bundle_purchase_notice	21		

Table 1 — How many records of each type. Every record in the file matched one of these, so nothing was left unrecognised.

The parser saves all of this to `data/transactions.json` as a list of dictionaries, which is exactly the shape the API needs in order to serve it.

3. API Endpoint Documentation

The API is written in plain Python using `http.server`, with no outside libraries. It gives you five endpoints covering the usual create, read, update and delete operations. The records live in memory in a dictionary keyed by id, which is why looking one up is quick no matter how many there are. Section 6 covers that properly. **Every one of these endpoints needs a username and password.**

Method	Endpoint	What it does	Success
GET	/transactions	Lists every transaction	200
GET	/transactions/{id}	Gets one transaction	200
POST	/transactions	Adds a new one	201
PUT	/transactions/{id}	Updates an existing one	200
DELETE	/transactions/{id}	Deletes one	200

Table 2 — The five endpoints.

3.1 Examples

GET /transactions/1 gets a single record.

```
$ curl -u admin:momo2025 http://localhost:8000/transactions/1
HTTP/1.0 200 OK
Content-Type: application/json

{
  "id": 1,
  "transaction_type": "incoming_money",
  "amount": 2000.0,
  "new_balance": 2000.0,
  "sender": "Jane Smith",
  "sender_phone": "*****013",
  "financial_transaction_id": "76662021700",
  "timestamp": "2024-05-10T16:30:51"
}
```

POST /transactions adds a record. You have to send `transaction_type` and `amount`, and the server picks the id.

```
$ curl -u admin:momo2025 -X POST http://localhost:8000/transactions \
-H "Content-Type: application/json" \
-d '{"transaction_type":"incoming_money","amount":7500,"sender":"Chol Deng"}'
HTTP/1.0 201 Created

{
  "id": 1692,
  "transaction_type": "incoming_money",
  "amount": 7500.0,
  "sender": "Chol Deng"
}
```

PUT /transactions/1692 changes a record. Send only the fields you want to change and the rest stay as they were.

```
$ curl -u admin:momo2025 -X PUT http://localhost:8000/transactions/1692 \
-H "Content-Type: application/json" -d '{"amount":9900}'
```

```
HTTP/1.0 200 OK
```

DELETE /transactions/1692 removes a record, and hands back what it deleted so you can see what has gone.

```
$ curl -u admin:momo2025 -X DELETE http://localhost:8000/transactions/1692
```

```
HTTP/1.0 200 OK
```

```
{ "message": "Transaction 1692 deleted", "deleted": { ... } }
```

And this is what you get with the wrong password.

```
$ curl -i -u admin:wrongpassword http://localhost:8000/transactions
```

```
HTTP/1.0 401 Unauthorized
```

```
WWW-Authenticate: Basic realm="MoMo Transactions API"
```

```
{ "error": "Unauthorized: invalid username or password", "status": 401 }
```

3.2 Error codes

Code	Name	When you get it
200	OK	A GET, PUT or DELETE worked
201	Created	A POST worked
400	Bad Request	Broken JSON, a missing required field, a field we do not recognise, or an amount that is negative or not a number
401	Unauthorized	No credentials, broken credentials, or the wrong password
404	Not Found	That id does not exist, or that URL is not one of ours

Table 3 — Error codes. Every error comes back in the same shape, `{"error": ..., "status": ...}`, so a client can handle them the same way each time. The full documentation is in `docs/api_docs.md`.

4. How We Implemented the Login

Every request has to carry an `Authorization` header holding the username and password joined with a colon and base64-encoded. The handler decodes that and checks both halves before any of the routing code runs, so there is no way to reach an endpoint without passing the check first.

```
Authorization: Basic YWRtaW46bW9tbzIwMjU=
```

Three details in our implementation are worth pointing out:

- We compare the credentials with `hmac.compare_digest()` instead of `==`. A normal comparison stops at the first character that does not match, so checking a nearly-correct password takes very slightly longer than checking a completely wrong one. That difference is tiny but measurable, and an attacker can use it to work out a password one character at a time. `compare_digest()` always takes the same time, so there is nothing to measure.
- The credentials are not written into the code. They come from the `API_USERNAME` and `API_PASSWORD` environment variables and only fall back to defaults for development, so the real password never has to be committed to git.
- Every way of failing returns a 401 rather than a crash. A missing header, a header that is not `Basic`, base64 that will not decode, and a simply wrong password are all handled, and all answered with a 401 plus a `WWW-Authenticate` header telling the client what to send.

5. Reflection: Why Basic Auth Is Weak

We are confident the Basic Auth above is implemented properly. The scheme itself is still the weakest part of the whole project, and it is worth being honest about why.

5.1 Base64 is not encryption

This is the main issue, and it is easy to miss because the word "encoded" sounds like it means something. It does not. Base64 uses no key and no secret, so anybody can undo it:

```
$ echo -n 'admin:momo2025' | base64
YWRtaW46bW9tbzIwMjU=

$ echo 'YWRtaW46bW9tbzIwMjU=' | base64 --decode
admin:momo2025
```

So the password is really being sent in plain text. Over plain HTTP, anybody who can see the traffic reads it straight off the wire. That might be someone else on the same coffee shop wifi, a router that has been tampered with, or the internet provider. Basic Auth is only acceptable at all if the connection is HTTPS, and even then the problems below still apply.

5.2 The other problems

- **The password goes out on every request.** HTTP does not remember anything between requests, so the client resends it each time. Load a page that makes thirty API calls and the password has crossed the network thirty times. Each one is a chance for it to end up in a proxy log or an error report, and because it is the real password rather than a temporary token, one leak is permanent.
- **It does not expire, and you cannot cancel it.** A Basic Auth password stays valid until a person changes it, and changing it is also the only way to shut it down. If five apps share the login, changing it breaks all five, including the four that were fine.
- **Everyone shares one account.** If a record gets deleted the log says "admin" did it, not which person, so there is no real audit trail. A read-only mobile app also ends up holding a password that

can delete every record in the database.

- **Nothing slows down guessing.** We have no rate limiting, no lockout after failed attempts and no second factor, so an attacker can try passwords as fast as the server answers them.

5.3 What we would use instead

JWT. The client sends its password once, to a login endpoint, and gets back a signed token that it sends from then on as `Authorization: Bearer <token>`. The token carries the user's id, their role and an expiry time, and it is signed with a key only the server knows. That fixes most of what is wrong above. The real password crosses the network once instead of constantly, tokens expire on their own so a stolen one is useful only briefly, and because the token says who the user is you can keep a proper audit log and give a read-only app a read-only token. The catch is that a JWT is signed but not encrypted, so anyone holding it can read what is inside and you must never put secrets in there. And since the server only checks the signature rather than looking anything up, you cannot easily cancel a token before it expires, which is why access tokens are usually kept short and paired with a refresh token.

OAuth 2.0. This is the right answer once other people's applications are involved. Instead of a third-party app holding the user's password, a separate authorization server issues tokens with scopes attached. A budgeting app that wants to show someone their MoMo spending could be given a token scoped to `transactions:read` and nothing else. It could list transactions and that is all, and the user could cut that one app off whenever they liked without changing their password or affecting anything else they use. The downside is that it is a lot more work and needs an authorization server running, which is overkill for a project with one trusted client.

Either way, the connection has to be HTTPS, because without it every scheme here leaks its password or token in transit. Passwords should be stored salted and hashed with something slow like bcrypt or Argon2, the login endpoint should be rate limited, and each person should have their own account with a role rather than everyone sharing one.

6. Comparing Linear Search and Dictionary Lookup

We built two ways of finding a transaction by its id in `dsa/dsa_comparison.py` and timed them against each other:

- **Linear search** walks the list of transactions one record at a time, checking each id until it finds the one it wants.
- **Dictionary lookup** builds a dictionary of id to transaction once, then fetches the record by key.

We ran both against all 1,691 records for **20 different ids** spread evenly across the dataset, repeating each measurement **1,000 times** with Python's `timeit` module. We also checked that both methods returned the same record every time, so we know we were comparing like with like. On top of the timings we counted how many record comparisons each method actually performs, done in a separate pass so the counting never slows down the timed run. That count turned out to be the more useful number, for reasons we get to below.

Target id	Position	Linear (µs)	Dict (µs)	Speed-up	Linear cmps	Dict cmps
1	1	0.079	0.063	1.3x	1	1
85	85	1.817	0.062	29.3x	85	1
169	169	3.572	0.064	56.1x	169	1
253	253	5.918	0.064	92.5x	253	1
337	337	8.143	0.065	125.3x	337	1
421	421	10.207	0.065	158.0x	421	1
505	505	12.131	0.066	184.0x	505	1
589	589	14.181	0.063	224.5x	589	1
673	673	16.685	0.064	262.6x	673	1
757	757	18.384	0.064	289.3x	757	1
841	841	20.464	0.062	328.5x	841	1
925	925	22.218	0.062	356.9x	925	1
1009	1009	24.351	0.064	379.0x	1009	1
1093	1093	26.555	0.062	425.7x	1093	1
1177	1177	28.546	0.063	451.9x	1177	1
1261	1261	30.529	0.064	476.1x	1261	1
1345	1345	33.045	0.064	512.7x	1345	1
1429	1429	34.741	0.063	551.1x	1429	1
1513	1513	36.469	0.063	575.1x	1513	1
1597	1597	39.065	0.063	616.8x	1597	1
AVERAGE		19.355	0.064	304.5x	799.0	1.0

Table 4 — What we measured. The full output is saved in `screenshots/dsa_results.txt`.

6.1 Why the dictionary is faster

The pattern in the table is hard to miss. Linear search gets steadily slower the further into the list the record sits, going from about 0.079 μ s for the very first record up to 39.065 μ s for one near the end. That is because it has to check every record it passes on the way, and that is what $O(n)$ means in practice. The dictionary stays at roughly 0.064 μ s the whole way down the table no matter where the record is, which is $O(1)$.

The reason is that a dictionary does not really search at all. Python runs the id through a hash function, and the answer tells it exactly which bucket the record is sitting in, so it goes straight there. It does the same amount of work whether the dictionary holds ten records or ten thousand. Linear search has no such shortcut. It has to compare records one by one until it gets a match.

The comparison counts show the same thing without depending on the computer at all, which is why we added them. Linear search needs exactly as many comparisons as the record's position: 1 for the first record, 1597 for the last one we sampled, and 799.0 on average across the twenty. The dictionary needs 1.0 comparison every single time. A faster laptop would shrink both timing columns, but it would not change these counts by one. The difference is in how the two methods work, not in how quick the machine is.

Averaged over the twenty ids the dictionary came out about **304.5x faster**. It is not free, though. You have to build the dictionary first, which is one pass through the data, and it keeps an extra reference for every record. You pay that once and get it back on every lookup afterwards, which is exactly why the API itself stores its transactions in a dictionary. It means getting, updating or deleting a record by id is an $O(1)$ operation instead of a scan.

6.2 What else could we have used?

A dictionary is not the only thing that beats a linear scan:

- **Binary search** on a list kept sorted by id. Each step throws away half of what is left, so it is $O(\log n)$. It needs no extra memory, but the list has to stay sorted, which means every insertion costs something.
- **A B-tree, or a database index.** This is what a real database uses for a primary key. It stays fast and it can also answer questions a hash cannot, like asking for every transaction between two dates.
- **A second dictionary on a different field**, say sender or transaction_type. Same trick, just applied to something other than the id, if that is what you need to search on.

7. Testing and Validation

We tested the API end to end with `curl`, using `screenshots/run_tests.sh`. It runs nine cases and saves the whole request and response transcript to `screenshots/api_test_results.txt`.

#	What we tested	Expected
1	GET /transactions with the right credentials	200
2	GET /transactions/1 with the right credentials	200
3	GET /transactions with the wrong password	401
4	GET /transactions with no credentials at all	401
5	POST /transactions creating a record	201
6	PUT on the record we had just created	200
7	DELETE the record we had just created	200

#	What we tested	Expected
8	GET that record again after deleting it	404
9	POST with an amount that is not a number	400

Table 5 — The nine test cases. All of them returned the status code we expected.

One small thing we changed part way through: the script now reads the new id out of the POST response instead of assuming what it will be. Before that, running the tests twice against the same server made the update, delete and 404 cases fail, because the id had moved on and the script was still looking for the old one.

8. Conclusion

The system reads all 1,691 SMS records out of the XML and turns them into structured JSON, serves them through five endpoints built on nothing but the Python standard library, and puts Basic Authentication in front of all of them, returning 401 for credentials that are missing, malformed or simply wrong. The search comparison gave us a real reason for a design decision rather than a guess: the dictionary was about 304.5x faster on average, and more importantly its cost does not grow as the dataset does.

The weakest part is the authentication, and that is worth saying plainly. Because base64 is encoding rather than encryption, the password is effectively sent in the clear on every request, it never expires, it cannot be revoked, and there is no way to tell one user from another. If this were going to be used for real, the next steps would be to put it behind HTTPS and move to short-lived JWTs, then add OAuth 2.0 scopes later if other people's applications ever needed access to the data.